

1. Описание условия задания

Первая разреженная (содержащая много нулей) матрица хранится в форме 3-х объектов (CSR):

- вектор A содержит значения ненулевых элементов;
- вектор IA содержит номера столбцов для элементов вектора A;
- вектор JA, в элементе N_k которого находится номер компонент в A и IA, с которых начинается описание строки N_k матрицы A.

Вторая разреженная матрица хранится в форме 3-х объектов (CSC):

- вектор B содержит значения ненулевых элементов;
- вектор JB содержит номера строк для элементов вектора A;
- вектор IB, в элементе N_k которого находится номер компонент в B и JB, с которых начинается описание столбцов N_k матрицы B.

Смоделировать операцию умножения двух матриц:

1. Хранящихся в указанной форме применяя алгоритм работы со сжатыми матрицами, с получением результата в форме хранения первой матрицы (CSR).
2. Хранящиеся в стандартной форме, применяя стандартный алгоритм работы с матрицами.
3. Сравнить время выполнения операций и объем памяти при использовании этих 2-х алгоритмов при различном проценте заполнения матриц.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные: матрицы в стандартном или разреженном виде, выбор действия в меню.

Формат файлов:

- Количество строк и столбцов матрицы,
- Элементы матрицы или три вектора разреженной матрицы.

Матрицы могут быть введены как с клавиатуры, так и из файла. Пользователю предоставляется выбор метода ввода.

Выходные данные:

- Матрица, являющаяся результатом умножения, выводится на экран.
- При умножении разреженных матриц результат выводится в CSR формате и, если размер небольшой ($\leq 30 \times 30$), в стандартном виде.
- При умножении стандартных матриц результат выводится в стандартном виде, если размер небольшой ($\leq 30 \times 30$).
- В результатах сравнения производительности: среднее время выполнения и объем памяти для обоих алгоритмов.

Ограничения:

- Количество строк/столбцов матрицы > 0 .

Способ обращения: консольный ввод/вывод.

Команда запуска программы: `./app.exe`. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- “ERROR_FILE” - Ошибка открытия файла.
- “ERROR_IO” - Ошибка чтения данных из файла или ввода с клавиатуры.
- “ERROR_INPUT” - Некорректный ввод (неверный формат, отрицательные размеры и т.д.).
- “ERROR_MEM” - Ошибка выделения памяти.
- “ERROR_SIZE” - Несовместимые размеры матриц для операции умножения.
- “ERROR_EMPTY_FILE” - Пустой файл.

Возможные действия, реализуемой программой:

0. Выход
1. Ввод CSR матрицы
2. Ввод CSC матрицы
3. Ввод стандартных матриц
4. Вывод CSR матрицы
5. Вывод CSC матрицы
6. Вывод CSR матрицы в стандартном виде
7. Вывод CSC матрицы в стандартном виде
8. Вывод стандартных матриц
9. Умножение разреженных матриц (CSR x CSC)
10. Умножение стандартных матриц
11. Сравнение производительности
12. Генерация файла с матрицей

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мною были реализованы следующие структуры данных:

```
c
typedef struct
{
    double* A;
    int* IA;
    int* JA;
    size_t num_elem;
    size_t lines;
    size_t columns;
} CSRMatrix;
```

Данная структура данных описывает разреженную матрицу в формате CSR. Поля структуры CSRMatrix:

- **A** – массив ненулевых элементов.
- **IA** – массив номеров столбцов для каждого элемента из A.
- **JA** – в элементе Nk которого находится номер компонент в A и IA, с которых начинается описание строки Nk матрицы A.
- **num_elem** – общее количество ненулевых элементов.
- **lines** – количество строк в матрице.
- **columns** – количество столбцов в матрице.

```
c
typedef struct
{
    double* B;
    int* JB;
    int* IB;
    size_t num_elem;
    size_t lines;
    size_t columns;
} CSCMatrix;
```

Данная структура данных описывает разреженную матрицу в формате CSC. Поля структуры CSCMatrix:

- **B** – массив ненулевых элементов.
- **JB** – массив номеров строк для каждого элемента из B.
- **IB** – массив, элементе Nk которого находится номер компонент в B и JB, с которых начинается описание столбцов Nk матрицы B.
- **num_elem** – общее количество ненулевых элементов.
- **lines** – количество строк в матрице.
- **columns** – количество столбцов в матрице.

Стандартные матрицы хранятся в виде динамического массива указателей на динамические массивы (double**).

4. Описание Алгоритмов

Основной алгоритм программы:

1. Вывести меню.
2. Считать выбор пользователя и проверить его корректности.
3. Выполнить соответствующего действия.
4. Либо снова вывести меню с ожиданием выбора, либо завершить программу с сообщением об ошибке.

Функции чтения (вывода) матриц в различных формах реализованы как поэлементное чтение (вывод) соответствующих элементов матриц.

Алгоритм умножения разреженных матриц (CSR x CSC):

1. Проверить совместимость размеров матриц:
 - a. Убедиться, что количество столбцов первой матрицы (CSR) равно количеству строк второй матрицы (CSC).
2. Выделить память под результирующую матрицу в формате CSR:
 - a. Инициализировать структуру результирующей матрицы.
 - b. Выделить память под массивы A, IA, JA.
 - c. Создать временный буфер для накопления строки результата.
3. Пройти по строкам первой матрицы (CSR):
 - a. Для каждой строки i от 0 до (количество_строк - 1):
 - i. Определить диапазон ненулевых элементов в текущей строке:
 1. Начало строки: JA[i], конец строки: JA[i+1].
 - b. Обработать каждый ненулевой элемент в строке i матрицы CSR:
 - i. Для каждого элемента в диапазоне [JA[i], JA[i+1]):
 1. Получить номер столбца $k = IA[\text{индекс}]$ и значение $\text{value_A} = A[\text{индекс}]$.
 - c. Для каждого столбца j матрицы CSC (от 0 до количество_столбцов - 1):
 - i. Найти элемент B[k][j] в столбце j матрицы CSC:
 - ii. Определить диапазон элементов в столбце j : IB[j] до IB[j+1].
 - iii. Найти элемент с индексом строки = k в массиве JB.
 - iv. Если элемент найден, выполнить умножение $\text{value_A} \times B[\text{индекс}]$.
 - v. Добавить результат во временный буфер в позиции j .
4. Перенести результат из временного буфера в результирующую матрицу:
 - a. Для каждого столбца j от 0 до (количество_столбцов - 1):
 - i. Добавить значение в массив A результирующей матрицы.
 - ii. Добавить номер столбца j в массив IA.
 - iii. Увеличить счетчик элементов результата.
 - b. Установить указатель на следующую строку в массиве JA.

5. Завершение операции:

- а. Установить фактическое количество ненулевых элементов в результате.
- б. Освободить временный буфер.
- с. Вернуть результирующую матрицу.

Алгоритм сравнения производительности:

1. Перенести матрицы из стандартного вида в CSR и CSC.
2. Инициализировать матрицы для результатов умножения.
3. Измерить время умножения для матриц в форме CSR/CSC.
4. Измерить время умножения для матриц в стандартной форме.
5. Вычислить средние результаты времени.
6. Вычислить используемую память.
7. Вывести результаты.

Реализуемые функции:

- // Функция для инициализации CSR матрицы

1. **void init_csr_matrix(CSRMatrix* matrix);**
- // Функция для инициализации CSC матрицы

2. **void init_csc_matrix(CSCMatrix* matrix);**
- // Создание матрицы

3. **double** allocate_matrix(size_t lines, size_t columns);**
- // Освобождение матрицы

4. **void free_matrix(double** matrix, size_t lines);**
- // Выделение памяти под CSR матрицу

5. **int allocate_csr_matrix(CSRMatrix* matrix, size_t lines, size_t columns, size_t num_elem);**
- // Освобождение CSR матрицы

6. **void free_csr_matrix(CSRMatrix* matrix);**
- // Создание CSC матрицы

7. **int allocate_csc_matrix(CSCMatrix* matrix, size_t lines, size_t columns, size_t num_elem);**
- // Освобождение CSC матрицы

8. **void free_csc_matrix(CSCMatrix* matrix);**
- // Ввод CSR матрицы с клавиатуры

9. **int input_csr_matrix(CSRMatrix* matrix);**
- // Загрузка CSR матрицы из файла

10. **int load_csr_matrix_from_file(CSRMatrix* matrix);**

```
// Вывод CSR матрицы
11. void print_csr_matrix(const CSRMatrix* matrix);

// Вывод CSR матрицы в виде обычной матрицы
12. void print_csr_normal(const CSRMatrix* matrix);

// Ввод CSC матрицы с клавиатуры
13. int input_csc_matrix(CSCMatrix* matrix);

// Загрузка CSC матрицы из файла
14. int load_csc_matrix_from_file(CSCMatrix* matrix);

// Вывод CSC матрицы
15. void print_csc_matrix(const CSCMatrix* matrix);

// Вывод CSC матрицы в виде обычной матрицы
16. void print_csc_normal(const CSCMatrix* matrix);

// Вывод стандартной матрицы
17. void print_matrix(double** matrix, size_t lines, size_t columns);

// Ввод стандартной матрицы
18. int input_matrix(double*** matrix, size_t* lines, size_t* columns);

// Загрузка стандартной матрицы из файла
19. int load_matrix_from_file(double*** matrix, size_t* lines, size_t* columns);

// Умножение разреженных матриц CSR × CSC
20. int multiply_csr_csc(const CSRMatrix *A, const CSCMatrix *B, CSRMatrix
    *result);

// Стандартное умножение плотных матриц
21. double** multiply_matrices(double** A, size_t A_lines, size_t A_columns,
    double** B, size_t B_lines, size_t B_columns);
```

// Преобразование стандартной матрицы в CSR
22. CSRMatrix normal_to_csr(double normal, size_t lines, size_t columns);**

// Преобразование стандартной матрицы в CSC
23. CSCMatrix normal_to_csc(double normal, size_t lines, size_t columns);**

// Сравнение производительности
24. void compare_performance(double A, size_t A_lines, size_t A_columns,
double** B, size_t B_lines, size_t B_columns);**

// Вывод меню
25. void print_menu(void);

// Вывод результата об ошибке
26. void print_error(int rc);

// Генерация стандартной матрицы
27. int generate_matrix(void);

5. Тесты

Позитивные тесты:

1. Попытка вывести матрицу (CSR, CSC, стандартные) до её ввода.
2. Попытка умножить матрицы (CSR, CSC, стандартные) до их ввода.
3. Попытка провести сравнение эффективности умножения до ввода стандартных матриц.
4. Выход из программы.
5. Ввод CSR матрицы 1x1 с помощью клавиатуры.
6. Ввод CSR матрицы 1x1 с помощью файла.
7. Ввод CSC матрицы 1x1 с помощью клавиатуры.
8. Ввод CSC матрицы 1x1 с помощью файла.
9. Ввод CSR матрицы 3x3 без ненулевых членов с помощью клавиатуры.
10. Ввод CSR матрицы 3x3 без ненулевых членов с помощью файла.
11. Ввод CSC матрицы 3x3 без ненулевых членов с помощью клавиатуры.
12. Ввод CSC матрицы 3x3 без ненулевых членов с помощью файла.
13. Ввод CSR матрицы 100x100 с помощью файла.
14. Ввод CSC матрицы 100x100 с помощью файла.
15. Ввод стандартных матриц 1x1 с помощью клавиатуры.
16. Ввод стандартных матриц 3x3 с помощью клавиатуры.
17. Ввод стандартных матриц 100x100 с помощью файла.
18. Вывод CSR матрицы в разреженной форме.
19. Вывод CSC матрицы в разреженной форме.
20. Вывод CSR матрицы в стандартной форме при кол-ве строк/столбцов < 30.
21. Вывод CSC матрицы в стандартной форме при кол-ве строк/столбцов < 30.
22. Вывод CSR матрицы в стандартной форме при кол-ве строк/столбцов > 30.
23. Вывод CSC матрицы в стандартной форме при кол-ве строк/столбцов > 30.
24. Вывод стандартных матриц при кол-ве строк/столбцов < 30.
25. Вывод стандартных матриц при кол-ве строк/столбцов > 30.
26. Умножение CSR на CSC матриц 1x1 на 1x1.
27. Умножение CSR на CSC матриц 1x3 на 3x1.
28. Умножение CSR на CSC матриц 100x100 на 100x100 при 100% заполненности.
29. Умножение стандартных матриц 1x1 на 1x1.
30. Умножение стандартных матриц 1x3 на 3x1.
31. Умножение стандартных матриц 100x100 на 100x100.

- | |
|---|
| <ul style="list-style-type: none">32. Сравнение эффективности при стандартных матрицах 1x1.33. Сравнение эффективности при стандартных матрицах 10x10.34. Сравнение эффективности при стандартных матрицах 100x100.35. Генерация файла с матрицей 1000 на 1000 с 100% заполнением.36. Генерация файла с матрицей 1 на 1 с 0% заполнением. |
|---|

Негативные тесты:
1. Пустой исходный файл. 2. Невалидный исходный файл. 3. Невалидный выбор меню. 4. Невалидный выбор режима ввода. 5. Невалидный ввод при вводе CSR матрицы. 6. Некорректные величины при вводе CSR матрицы. 7. Невалидный ввод при вводе CSC матрицы. 8. Некорректные величины при вводе CSC матрицы. 9. Невалидный ввод при вводе стандартных матрицы. 10. Некорректные величины при вводе стандартных матрицы. 11. Некорректные размерности матриц для умножения. 12. Некорректный ввод файла с матрицей.

5. Оценка эффективности

Количество запусков алгоритма для измерения времени и вычисления среднего:

- 10 раз для матриц 100x100 и 500x500 элементов;
- 5 раз для матриц 1000x1000 элементов.

Процент заполнения	Размер матриц	Время стандартного (с)	Время разреженного (с)	Отношение времени (Станд/Разр)	Память стандартная (байт)	Память разреженная (байт)	Отношение памяти (Станд/Разр)
0%	100×100	0.003259	0.000037	88.09	160,000	808	198.02
3%	100×100	0.003234	0.000491	6.59	160,000	8,008	19.98
5%	100×100	0.003098	0.001161	2.67	160,000	12,808	12.49
10%	100×100	0.002580	0.001947	1.32	160,000	24,808	6.45
12%	100x100	0.002565	0.002529	1.01	160,000	27,208	5.88
20%	100×100	0.002574	0.006676	0.39	160,000	48,808	3.28
30%	100×100	0.002580	0.010045	0.26	160,000	72,808	2.20
50%	100×100	0.002581	0.021048	0.12	160,000	120,808	1.32
66%	100x100	0.002540	0.031607	0.08	160,000	159196	1.01
70%	100×100	0.002581	0.036135	0.07	160,000	180,796	0.88
100%	100×100	0.002583	0.053182	0.05	160,000	240,784	0.66
0%	500×500	0.373172	0.000623	598.70	4,000,000	4,008	998.00
3%	500×500	0.375192	0.078710	4.77	4,000,000	184,008	21.74
5%	500×500	0.372527	0.192959	1.93	4,000,000	303,960	13.16
7%	500x500	0.377387	0.322191	1.17	4,000,000	423,984	9.43
8%	500x500	0.374322	0.401666	0.93	4,000,000	483,984	8.26
10%	500×500	0.372558	0.596695	0.62	4,000,000	603,984	6.62
20%	500×500	0.371391	1.972523	0.19	4,000,000	1,203,912	3.32
30%	500×500	0.371748	3.951278	0.09	4,000,000	1,803,936	2.22
50%	500×500	0.371665	9.348859	0.04	4,000,000	3,003,888	1.33
66%	500x500	0.373563	14.501354	0.02	4,000,000	3963852	1.01
70%	500×500	0.371621	15.758422	0.02	4,000,000	4,203,768	0.95
100%	500×500	0.372826	24.595735	0.02	4,000,000	6,003,672	0.67
0%	1000×1000	3.344931	0.001088	3075.33	16,000,000	8,008	1998.00
3%	1000×1000	3.497604	1.038561	3.37	16,000,000	727,984	21.98
5%	1000×1000	3.516361	2.429881	1.45	16,000,000	1,207,984	13.25
6%	1000x1000	3.534823	3.284673	1.07	16,000,000	1,447,924	11.05
7%	1000x1000	3.521565	4.322135	0.81	16,000,000	1,687,876	9.48
10%	1000×1000	3.592812	8.171601	0.44	16,000,000	2,407,912	6.64
50%	1000×1000	3.552838	143.469844	0.02	16,000,000	12,007,312	1.33
66%	1000x1000	3.507463	213.453252	0.01	16,000,000	15,848,008	1.00
70%	1000×1000	3.475008	243.639804	0.01	16,000,000	16,807,096	0.95
100%	1000×1000	3.518833	380.825059	0.01	16,000,000	24,006,784	0.67

Эти результаты демонстрируют, что разреженные алгоритмы эффективны по времени для матриц с разреженностью меньше 7-10%, а по памяти – меньше 65%, а для менее разреженных матриц стандартные подходы предпочтительнее.

6. Выводы

Итак, можно сделать вывод, что использовать алгоритмы для работы с разреженными матрицами эффективно для матриц, имеющих $< 7-10\%$ заполнения. Это подтверждают данные:

Разреженные алгоритмы демонстрируют экстремальное преимущество при работе с сильно разреженными матрицами (0-5% заполнения), особенно для больших размеров.

Критический порог эффективности снижается с ростом размера матрицы: для 1000×1000 матриц разреженные методы эффективны до 8% заполнения, для 100×100 - до 12%.

Экономия памяти более стабильна, чем выигрыш по времени, и сохраняется до более высоких процентов заполнения (66%).

7. Ответы на контрольные вопросы

1. Что такое разреженная матрица, какие схемы хранения таких матриц Вы знаете?

Разреженная матрица - это матрица, в которой значительная часть элементов равна нулю. В таких матрицах количество ненулевых элементов обычно выражается как $n[g]$, где $g < 1$, и для матриц порядка n типичные значения g лежат в интервале 0.2...0.5.

Известные схемы хранения разреженных матриц:

- 1.1. **Линейный связный список** - последовательность ячеек, содержащих элемент и указатель на следующую ячейку.
- 1.2. **Кольцевой связный список** - модификация линейного списка.
- 1.3. **Двунаправленные стеки и очереди**
- 1.4. **Диагональная схема хранения** - для симметричных матриц.
- 1.5. **Связная схема Кнута** - избыточная схема, позволяющая легко осуществлять любые операции:
 - 1.5.1. AN - массив значений элементов.
 - 1.5.2. I, J - массивы индексов строк и столбцов.
 - 1.5.3. NR, NC - номера следующих элементов по строке и столбцу.
 - 1.5.4. JR, JC - указатели для входа в строку и столбец.
- 1.6. **Разреженный строчный формат (CSR)** - наиболее широко используемая схема:
 - 1.6.1. AN - значения ненулевых элементов.
 - 1.6.2. JA - столбцовые индексы.
 - 1.6.3. IA - указатели начала строк.

2. Каким образом и сколько памяти выделяется под хранение разреженной и обычной матрицы?

Обычная матрица:

- Память: $n \times m \times \text{sizeof}(\text{double})$.
- Сложность: $O(n \times m)$.

Разреженная матрица (CSR формат):

- A[]: $NNZ \times \text{sizeof}(\text{double}) = NNZ \times 8$ байт.
- IA[]: $NNZ \times \text{sizeof}(\text{int}) = NNZ \times 4$ байт.
- JA[]: $(n + 1) \times \text{sizeof}(\text{int}) = (n + 1) \times 4$ байт.
- Общий объем: $12 \times NNZ + 4 \times (n + 1)$ байт.

3. Каков принцип обработки разреженной матрицы?

Принцип обработки разреженной матрицы заключается в выполнении операций только с ненулевыми элементами, что позволяет:

- Сократить количество арифметических операций.
- Уменьшить объем используемой памяти.
- Повысить эффективность вычислений для больших матриц.

4. В каком случае для матриц эффективнее применять стандартные алгоритмы обработки матриц? От чего это зависит?

Стандартные алгоритмы эффективнее при:

- Высоком проценте заполнения (>7-10% ненулевых элементов).
- Небольших размерах матриц.

Эффективность зависит от:

1. Плотности матрицы - критический порог по экспериментальным данным:
 - a. По времени: 7-10% заполнения.
 - b. По памяти: 65% заполнения.
2. Размера матрицы - для больших матриц преимущества разреженных методов проявляются сильнее.